



【Dify×LINE】DifyとLINEを連携させよう GASコード付き！【エージェント編】



AI BOOTCAMP 公式note

2024年6月27日 14:12

購入済

SNS等で話題になっているノーコードツールの「Dify」

「Difyで作成したエージェント・チャットボット・ワークフローをLINEで使用できたらな〜」なんて思ったりしませんか？

今回の記事は、Difyで作成した「エージェント」「チャットボット」をLINEと連携させて、**LINEからエージェントを動かす**までの連携手順を説明します。

GASを使用した連携になりますが、APIキーやトークンを入力するだけの完成されたGASコードもプレゼント。

これを使うことで簡単にLINEとの連携が可能です！

Difyの概要や活用事例も解説していますので、こちらも是非ご覧ください。

また、「Difyは知っているけど、なにから作成して良いかわからない」という方のために、「**yamlファイル付きのDifyを活用したニュースアプリの作成手順**」の有料記事も手頃な価格で読むことができ、**Difyを使用した「時短」を体験**できるかと思います。

【Dify】情報収集を完全自動化！HTTPリクエストでニュースアプリを作ろう すぐに使えるyaml付き！

ビジネスの現場では、必要なニュースやトレンド情報を素早く正確に把握することが欠かせません。しかし、毎日多くのウェブサイトをチェックする作業は手間がかかり、生産性を下げてしまう要因となりがちです。そこで注目したいの...

♡ 48



AI BOOTCAMP 公式note
2024/06/20 23:35

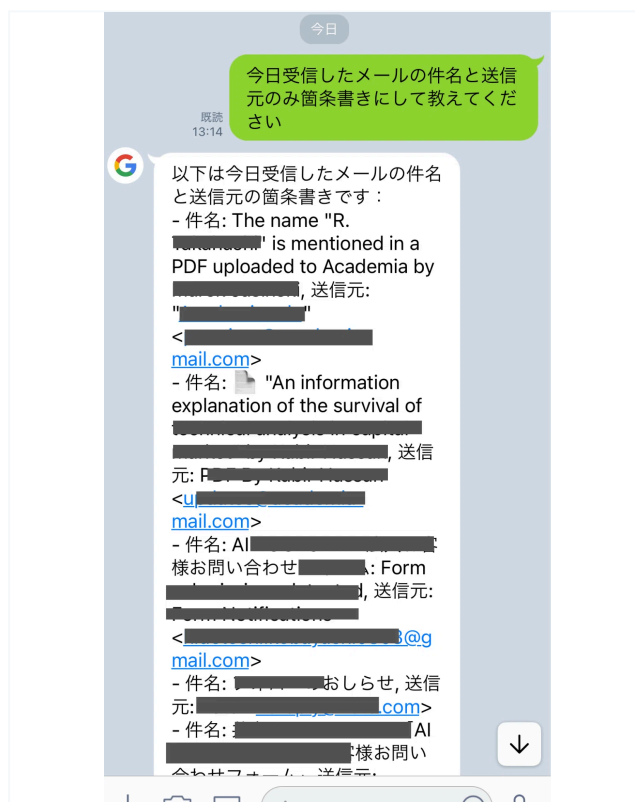
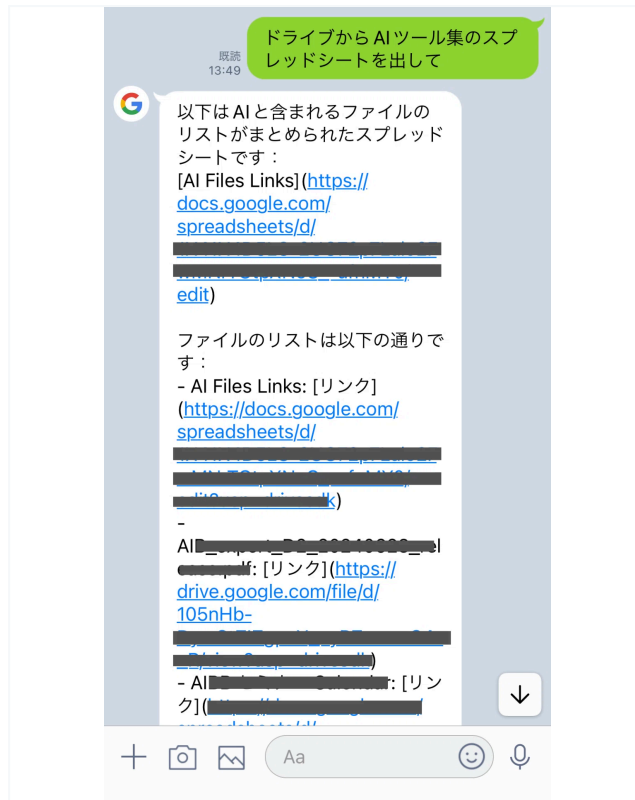


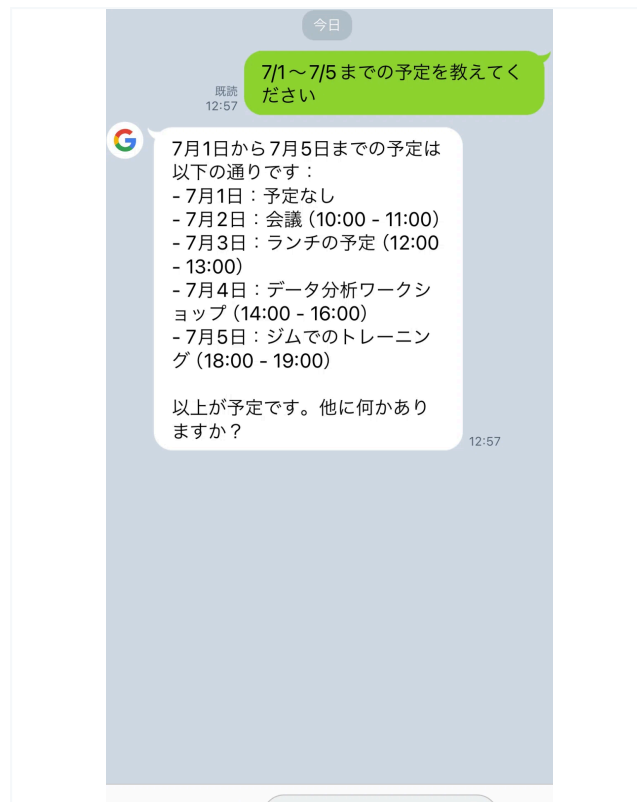
【Dify】
情報収集を完全自動化！
HTTPリクエストで
ニュースアプリを作ろう
すぐに使えるyaml付き！

© AI BOOTCAMP

DifyエージェントとLINE連携の概要と仕組み

完成形は以下の画像のようになります。





これはDifyで作成した、Googleサービスで指示した内容を表示してくれるエージェントになります。

LINEとの連携の仕組みはシンプルです。

- ユーザーからのLINEメッセージがエージェントへのプロンプトになる
- エージェントからの回答をLINEに反映させる

エージェントをLINEを連携させることによって、外出先からでもDifyエージェントを操作できますし、LINEで完結できるので「時短」にもなります。

DifyエージェントとLINEを連携させるには以下のアカウント必要となります。

- Difyのエージェント(ご自身で作成したもの)
- Googleアカウント
- LINEアカウント

有料部分について

拡散していただくと【無料】で読むことができます！

有料記事の部分では「**Google App Script(GAS)を使用してのDifyエージェントとLINEの連携**」をお話いたしますので、**Difyでエージェントを作成している方が対象**になります。
知識がある方は既に把握している内容になるかもしれませんが、有料記事購入は必ず自己判断でお願いいたします。

また、**DifyエージェントでOAuth認証などが必要な設定をしている場合は、今回の手順だとLINEからの認証はブロックされます**ので、こちらでご自身のDifyエージェントに合わせて判断をお願いいたします。

※返金是不可能的なのでご注意ください

初心者の方が理解しやすいよう、細かく説明いたします。

※ 更新情報

・2024年12月16日

LINEの仕様変更により方法が変わったため、有料部分の記事をアップデートしました。

----- ここから先は有料部分です -----

▼ 目次

DifyエージェントとLINE連携の概要と仕組み

有料部分について

LINE連携までの流れ

1. DifyエージェントのAPIキー取得

2. GASでコード作成

3. LINEチャネルアクセストークンの取得

3-1. LINE Official Account Managerにログイン

3-2. アカウントの作成

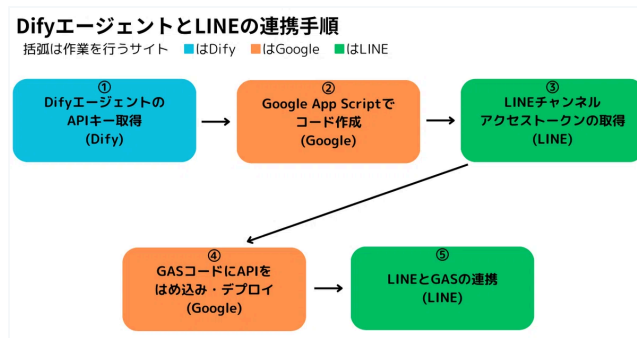
3-3. プロバイダーを作成

3-4. チャネルアクセストークンを取得

すべて表示

LINE連携までの流れ

DifyエージェントとLINEの連携は以下の流れになります。
順番に説明していきます。



1. DifyエージェントのAPIキー取得
2. Google App Script(以下GAS)でコード作成
3. LINEチャンネルアクセストークンの取得
4. GASコードにAPIをはめ込み・デプロイ
5. LINEとGASの連携

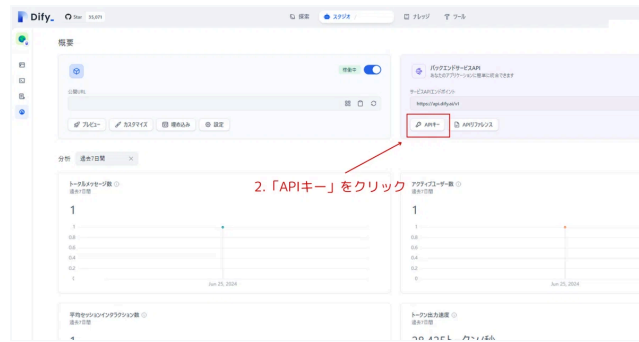
1. DifyエージェントのAPIキー取得

最初にDifyエージェントのAPIキーを取得します。

- 1-1. 作成しているDifyエージェントのページを開き、「概要」をクリック



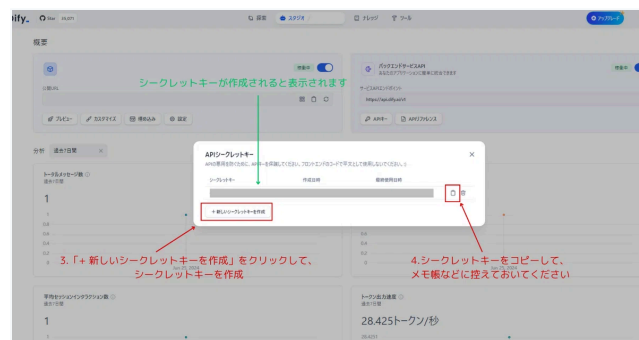
- 1-2. 「バックエンドサービスAPI」にある「APIキー(鍵マーク)」をクリック



1-3. 「+ 新しいシークレットキーを作成」をクリックして、シークレットキーを作成

1-4. シークレットキーをコピーして、メモ帳などに控えておいてください

※作成したシークレットキーは外部に漏れないように厳重に管理してください



これで1.DifyエージェントのAPIキー取得は終わりです。

2. GASでコード作成

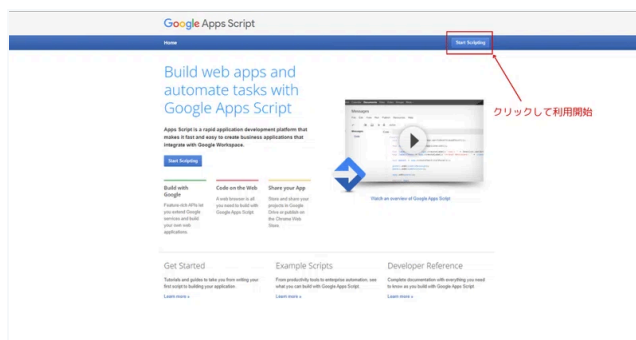
次にGASでLINEと連携させるためのコードを作成します

2-1. 以下のリンクから**Google App Script**にアクセスして利用を開始します
(無料で利用できます)

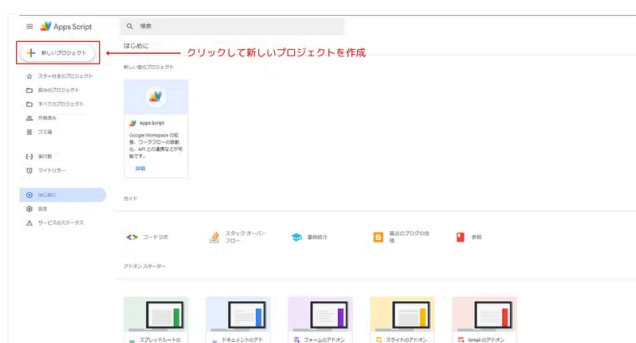
Apps Script – Google Apps Script

www.google.com/apps-script/

「Start Scripting」をクリック



画面左上の「新しいプロジェクト」をクリックしてプロジェクトを作成



プロジェクト名はわかりやすいように「Dify_Agent_LINE」に変えておきましょう

2-2. 「コード.gs」に以下のコードを貼り付け



※ 2024年9月12日追記

こちらは『Difyエージェント』『Difyチャットボット』との連携では実装確認できおりますが、ワークフロー等では稼働できません。予めご了承ください。

```
/**
 * このスクリプトは、LINEからのメッセージを受信し、Dify APIで処理し、応答をLINEに送り返します。
 */

// 定数(ご自身の環境に合わせて変更してください)
const DIFY_API_KEY = 'DIFY_API_KEY'; // Dify APIキーを入力
const LINE_ACCESS_TOKEN = 'LINE_ACCESS_TOKEN'; // LINEチャンネルアクセストークンを入力
const DIFY_API_URL = 'https://api.dify.ai/v1/chat-messages'; // Dify APIのエンドポイント
const LINE_REPLY_URL = 'https://api.line.me/v2/bot/message/reply'; // LINE Messaging

/**
 * LINEウェブフックからのPOSTリクエストを処理します
 * @param {Object} e - LINEからのイベントオブジェクト
 */
function doPost(e) {
  try {
    const json = JSON.parse(e.postData.contents);
    const event = json.events[0];
    const userMessage = event.message.text;
    const replyToken = event.replyToken;
    const userId = event.source.userId;

    if (!replyToken || !userMessage) {
      Logger.log("エラー： 無効なイベントデータ");
      return;
    }

    const difyResponse = sendChatMessage(userMessage, userId);
    replyToLine(replyToken, difyResponse || "申し訳ありません。メッセージを処理できません。");
  } catch (error) {
    Logger.log("doPost関数でエラーが発生: " + error.toString());
  }
}

/**
 * Dify APIにチャットメッセージを送信し、応答を処理します
 * @param {string} userMessage - ユーザーからのメッセージ
 * @param {string} userId - ユーザーID
 * @returns {string} Difyからの処理された応答
 */
function sendChatMessage(userMessage, userId) {
  const options = {
    method: 'post',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer ' + DIFY_API_KEY
    },
    payload: JSON.stringify({
      inputs: {},
      query: userMessage,
      response_mode: 'streaming',
      user: userId
    }),
    muteHttpExceptions: true
  };
}
```

```
try {
  const response = UrlFetchApp.fetch(DIFY_API_URL, options);
  return parseDifyResponse(response.getContentText());
} catch (e) {
  Logger.log("sendMessage関数でエラーが発生: " + e.toString());
  return null;
}
}

/**
 * Dify APIからのストリーミングレスポンスを解析します
 * @param {string} responseText - Dify APIからの生のレスポンステキスト
 * @returns {string} 解析された応答
 */
function parseDifyResponse(responseText) {
  let result = "";
  const lines = responseText.split("\n");

  for (const line of lines) {
    if (line.startsWith("data: ")) {
      const data = JSON.parse(line.substring(6));
      if (data.event === "agent_message" || data.event === "message") {
        result += data.answer;
      } else if (data.event === "message_end") {
        return result;
      } else if (data.event === "error") {
        Logger.log("Difyからエラーイベントを受信: " + data.message);
        return "エラー: " + data.message;
      }
    }
  }
  return result;
}

/**
 * LINEに返信メッセージを送信します
 * @param {string} replyToken - LINEからの返信トークン
 * @param {string} message - 送信するメッセージ
 */
function replyToLine(replyToken, message) {
  const payload = {
    replyToken: replyToken,
    messages: [{
      type: 'text',
      text: message
    }]
  };

  const options = {
    method: 'post',
    contentType: 'application/json',
    headers: {
      'Authorization': 'Bearer ' + LINE_ACCESS_TOKEN
    },
    payload: JSON.stringify(payload)
  };

  try {
```

```

    UrlFetchApp.fetch(LINE_REPLY_URL, options);
  } catch (error) {
    Logger.log("replyToLine関数でエラーが発生: " + error.toString());
  }
}

/**
 * スクリプトの初期設定を確認します
 * この関数は手動で実行してください
 */
function checkSetup() {
  if (!DIFY_API_KEY) {
    Logger.log("エラー: Dify APIキーが設定されていません。");
  }
  if (!LINE_ACCESS_TOKEN) {
    Logger.log("エラー: LINEアクセストークンが設定されていません。");
  }
  Logger.log("設定の確認が完了しました。エラーがない場合は、スクリプトを使用する準備が整っています");
}

```

以下は貼り付けしたGASコードの説明になります。

メイン関数：doPost

この関数は、LINEからメッセージを受け取って処理します。

1. メッセージの内容、返信トークン、ユーザーIDを取得します。
2. 取得したメッセージをDify APIに送信し、応答を受け取ります。
3. Dify APIからの応答をLINEに返します。

```

function doPost(e) {
  var json = JSON.parse(e.postData.contents); // 受信したデータをJSON形式に変換
  var userMessage = json.events[0].message.text; // ユーザーのメッセージを取得
  var replyToken = json.events[0].replyToken; // 返信トークンを取得
  var userId = json.events[0].source.userId; // ユーザーIDを取得
  var conversationId = ""; // 会話IDを初期化

  // Dify APIにメッセージを送信し、応答を取得
  var difyResponse = sendChatMessage(userMessage, conversationId, userId);
  // Dify APIからの応答をLINEに返信
  replyToLine(replyToken, difyResponse);
}

```

Dify APIにメッセージを送信して応答を取得する関数：sendChatMessage

1. ユーザーのメッセージをDify APIに送信します。
2. HTTP POSTメソッドを使ってリクエストを送信し、応答を受け取ります。
3. 受け取った応答を解析し、必要な情報を取り出します。

```
function sendMessage(userMessage, conversationId, userId) {
  var options = {
    'method': 'post', // メッセージ送信にはPOSTを使用
    'headers': {
      'Content-Type': 'application/json', // JSON形式でデータを送信
      'Authorization': 'Bearer ' + DIFY_API_KEY // 認証のためのAPIキーを設定
    },
    'payload': JSON.stringify({
      'inputs': {}, // 特定の入力データ(空のオブジェクト)
      'query': userMessage, // ユーザーのメッセージを設定
      'response_mode': 'streaming', // ストリーミングモードで応答を取得
      'user': userId, // ユーザーIDを設定
      'conversation_id': conversationId // 会話IDを設定
    }),
    'muteHttpExceptions': true // HTTP例外を無視する設定
  };

  try {
    var response = UrlFetchApp.fetch(DIFY_API_URL + "chat-messages", options); // Dify APIにリクエストを送信
    var result = "";
    var lines = response.getContentText().split("\n"); // 応答を行単位に分割

    for (var i = 0; i < lines.length; i++) {
      var line = lines[i];
      if (line.startsWith("data: ")) {
        var data = JSON.parse(line.substring(6)); // "data: "以降の部分をJSONとして解析
        if (data.event === "agent_message") {
          result += data.answer; // 応答を結果に追加
        } else if (data.event === "message_end") {
          // レスポンスから新しいconversation_idを取得
          conversationId = data.conversation_id;
          return result; // 応答を返す
        } else if (data.event === "error") {
          return "Error: " + data.message; // エラーメッセージを返す
        }
      }
    }
  } catch (e) {
    Logger.log(e.toString()); // エラーをログに記録
    return 'An error occurred while processing the request.'; // エラーメッセージを返す
  }

  return result; // 最終的な応答を返す
}
```

LINEに応答を返す関数：replyToLine

1. LINEのAPIエンドポイントURLに、ユーザーに返信するためのメッセージを送信します。

```
function replyToLine(replyToken, message) {  
  var url = 'https://api.line.me/v2/bot/message/reply'; // LINE APIのURL  
  var payload = {  
    'replyToken': replyToken, // 返信トークンを設定  
    'messages': [{  
      'type': 'text', // メッセージタイプをテキストに設定  
      'text': message // 応答メッセージを設定  
    }]  
  };  
  
  var options = {  
    'method': 'post', // POSTメソッドを使用  
    'contentType': 'application/json', // JSON形式でデータを送信  
    'headers': {  
      'Authorization': 'Bearer ' + LINE_ACCESS_TOKEN // 認証のためのLINEアクセストークンを設  
    },  
    'payload': JSON.stringify(payload) // リクエストペイロードをJSON形式に変換  
  };  
  
  UrlFetchApp.fetch(url, options); // LINE APIにリクエストを送信  
}
```

このコードは、LINEから受信したメッセージをDify APIに送信し、その応答をLINEに返す処理を行います。各関数は、それぞれ特定の役割を持っており、順番に実行されることでこの流れを実現しています。

「コード.gs」にコードを貼り付けたら、2.GASでコード作成は終了になりますが、この後にDifyのAPIキーとLINEのアクセストークンの貼り付けがありますので、画面は閉じないでください。

3. LINEチャネルアクセストークンの取得

※ 2024年12月16日追記

LINEの仕様変更により、LINEチャネルアクセストークンの取得方法が変わったため、記事をアップデートしました。

次はLINEの設定・チャネルアクセストークンの取得を行います

3-1. LINE Official Account Managerにログイン

以下の**LINE Official Account Manager**にログインします。

※「ビジネスアカウント」でログインはできますが、今回は個人の「LINEアカウント」でログインします

LINE Business ID

manager.line.biz

ログインすると自動で**アカウントリスト**の画面に飛びます

3-2. アカウントの作成

次にアカウントの作成を行います。左側にある「**作成**」をクリック。



LINE公式アカウントの作成画面で必要項目を入力します。

1 会社・店舗情報を登録 2 入力内容の確認 3 申し込み完了

LINE公式アカウントの作成

ログイン情報

ユーザー名 ログアウト

サービス対象国・地域 日本 日本の全主要アジアが適用されます。

アカウント情報

アカウント名 ① 例: Brownのファン

メールアドレス

会社・事業者の所在国・地域 日本

会社・事業者名

業種 ② 大業種を選択 小業種を選択

運用目的 (複数選択可) ③

- ☐ お店を知ってもらいたい、興味を持ってもらいたい
- ☐ 来店してもらいたい、リピーターになってもらいたい
- ☐ 潜在顧客・見込み顧客を集めたい
- ☐ お問い合わせに対応したい、お客さんとチャットしたい
- ☐ ホームページ・SNSの代算として使いたい
- ☐ サイトに人を集めたい、登録してもらいたい
- ☐ その他

主な使い方 ④

- ☐ メッセージ配信用
主にメッセージ配信を利用して、友だちに情報を発信する。
- ☐ チャット・LINEコール用
主にチャットやLINEコールを利用して、友だちからの問い合わせに対応する。
- ☐ 上記以外の使い方
- ☐ 未定
この選択によって利用可能な機能が制限されることはありません。

LINE公式アカウント利用規約

上記の利用規約にご同意の上、確認をクリックしてください。

確認

- ① お好きな名前を入力してください
- ② 大業種：個人、小業種：個人(その他)でかまいません
- ③ 一番下の**その他**でかまいません
- ④ 一番下の**未定**でかまいません

必須項目の入力が終わったら、確認を押し、次のページで完了をクリックします。

LINE公式アカウントが作成されました。

以下の内容でアカウントが作成されました。
LINE公式アカウントのサービスを利用する場合は、LINE Official Account Managerにログインしてください。

LINE Official Account Managerへ

アカウント情報

アカウント名 テスト

業種 個人・個人(その他)

ベーシックID [Redacted]

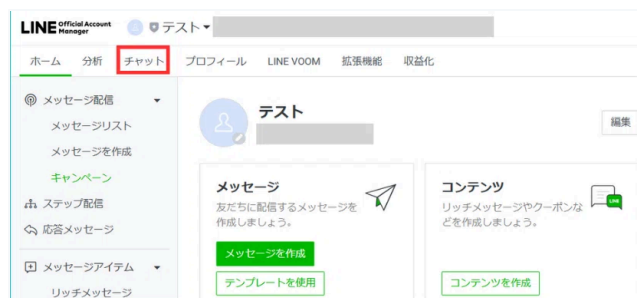
よくある質問

ご不明な点がございましたら、以下のリンクをご確認ください。
https://help2.line.me/official_account_lp/web/

この画面になったら完成

3-3. プロバイダーを作成

Message APIを使用するために、**プロバイダー**を作成します。上の画像の「LINE Official Account Managerへ」を押し、同意を2回押して、作成したLINE公式アカウントのホーム画面に移動します。



移動したら、上のメニューバーにある「チャット」をクリックして、応答設定に移動します。

左側のメニューバーにある「**Message API**」を押して移動し、「**Message APIを利用する**」をクリックします。

以下のポップアップが出てくるので、お好きな名前を付けて、プロバイダーを作成してください。

- ・プライバシーポリシーURL：ご自身のみで使用する場合は不要
- ・利用規約URL：ご自身のみで使用する場合は不要

プロバイダーを選択

×

アカウントを管理するプロバイダー（企業または個人）を選択してください。

プロバイダーとは、サービスを提供し、ユーザーの個人情報を取得する開発者個人、企業、または団体等のことを指します。
詳細は[LINE Developers](#)をご確認ください。

● プロバイダーを作成

プロバイダー名を入力

0/100

「[LINE公式アカウントAPI利用規約](#)」にご同意の上、下の[同意する]をクリックしてください。

キャンセル

同意する

3-4. チャネルアクセストークンを取得

次に、チャネルアクセストークン(長期)を取得します。以下の**LINE Developers**にログインします。

<https://note.com/aibootcamp/n/nca1d5623e6f9>

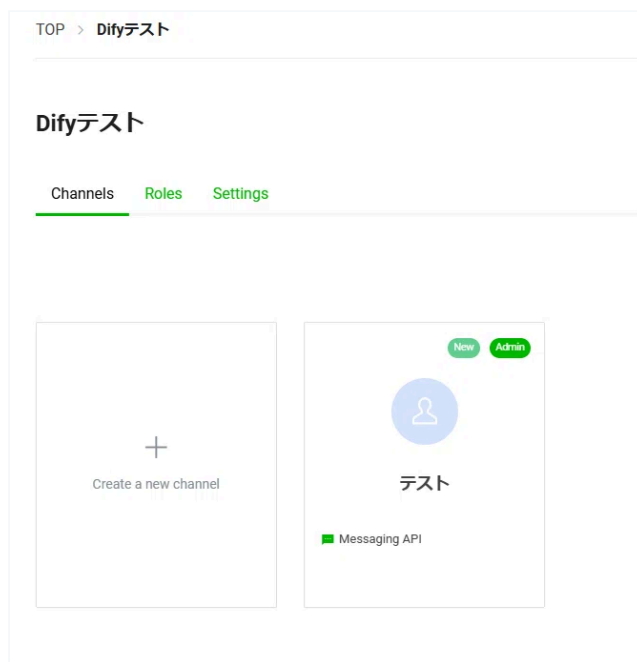
16/26

LINE Developers

LINE Developersサイトは開発者向けのポータルサイトです。LINEプラットフォームのさまざまな開発者向けプロダ

developers.line.biz

すると、左側のメニューバーのAdminの部分に、先ほど作ったプロバイダーが表示されていると思います。プロバイダー名をクリックし、表示されているLINE公式アカウントをクリックしてください。



「チャンネル基本設定」の右隣の「**Messaging API**」をクリックします（QRコードが表示されている画面です）



「Messaging API」の画面一番下の「**チャンネルアクセストークン（長期有効）**」の発行もしくは問題を

クリックしてください。
クリックすると**チャンネルアクセストークン（長期有効）**が発行されますので、こちらもDifyエージェントAPIと同じようにコピーしてメモ帳に控えておいてください。

※LINEチャンネルアクセストークンも外部に漏れないように厳重に管理してください



コピーしたチャンネルアクセストークンをメモ帳などに控えたら、3.LINEチャンネルアクセストークンの取得の設定は終了になります

4. GASコードにAPIをはめ込み・デプロイ

次は先程控えたDifyエージェントAPIキーと、LINEチャンネルアクセストークンをGASコードに挿入して、デプロイを行います。

再びGoogle App Scriptに移ります。

4-1. DIFY_API_KEYの設定

「コード.gs」に貼り付けしたコードの一番上の、`var DIFY_API_KEY = 'DIFY_API_KEY'`の「`DIFY_API_KEY`」を先程控えたDifyエージェントAPIキーに変更します

※必ず「`'`」の間に入れてください！

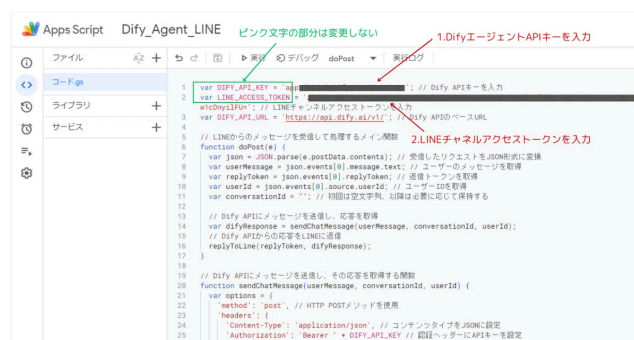
「`'`」を消すと認識されないので注意してください

4-2. LINE_ACCESS_TOKENの設定

次にコード上から二番目の、`var LINE_ACCESS_TOKEN = 'LINE_ACCESS_TOKEN'`の

「`LINE_ACCESS_TOKEN`」も先程控えたLINEチャンネルアクセストークンに変更します

※こちらも必ず「`'`」の間に入れてください



4-3. デプロイ

DifyエージェントAPIキーとLINEチャネルアクセストークンを入力したら、「**デプロイ**」を行います。その前に「**デプロイ**」について説明します。

デプロイの説明

1. デプロイとは？

デプロイは、開発したソフトウェアやアプリを実際に使えるようにすることです。作ったものをユーザーが使える状態にするための最後のステップと考えてください。

2. 使い方

デプロイするには、開発したソフトウェアをサーバーやクラウドにアップロードし、それをユーザーがアクセスできるように設定します。これにはいくつかの手順があり、使うツールや環境によって異なります。

3. デプロイで出来ること

デプロイを行うと、以下のことができます

- ・ **ユーザーに公開**：開発したソフトウェアをインターネット上で公開し、ユーザーがアクセスして使えるようにします。
- ・ **更新の反映**：ソフトウェアに新しい機能や修正を加えた場合、それをデプロイすることでユーザーが最新バージョンを使えるようにします。
- ・ **テストの実施**：デプロイして実際の環境でテストを行い、問題がないか確認します。

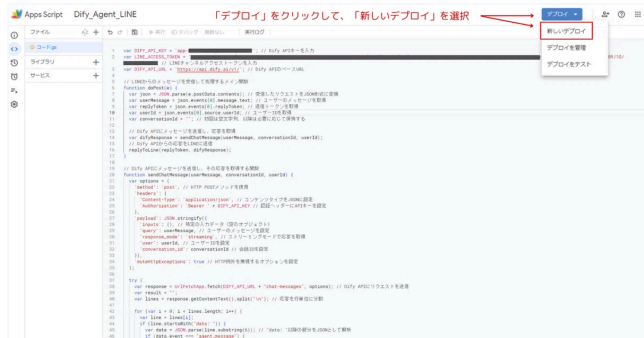
4. デプロイの手順・準備

ソフトウェアが正しく動作することを確認し、必要なファイルやデータを整理します。

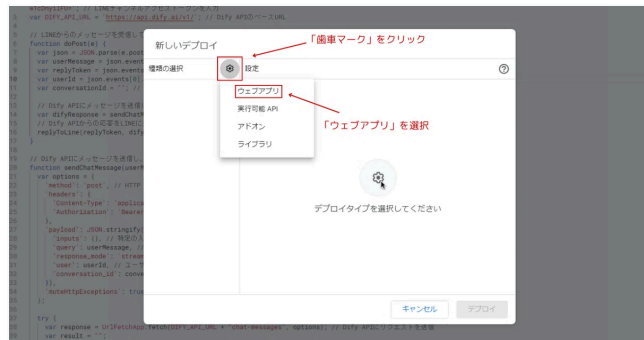
- ・ **アップロード**：サーバーやクラウドにソフトウェアをアップロードします。
- ・ **設定**：ソフトウェアが正しく動作するように設定を行います。テスト：実際の環境で動作を確認し、問題がないかテストします。
- ・ **公開**：ユーザーがアクセスできるようにソフトウェアを公開します。

デプロイは、開発したソフトウェアやアプリをユーザーが使えるようにするための重要なステップです。ウェブサイトやアプリをインターネット上に公開し、更新を反映させるために必要な作業です。

画面右上の「**デプロイ**」をクリックして、「**新しいデプロイ**」を選択。



「種類の選択」の右隣の「歯車マーク」をクリックして、一番上の「ウェブアプリ」を選択。

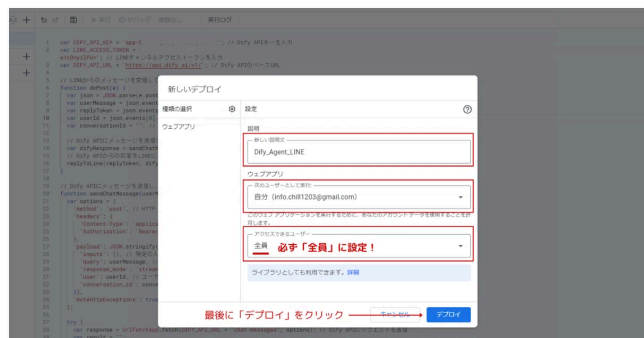


設定は上から、

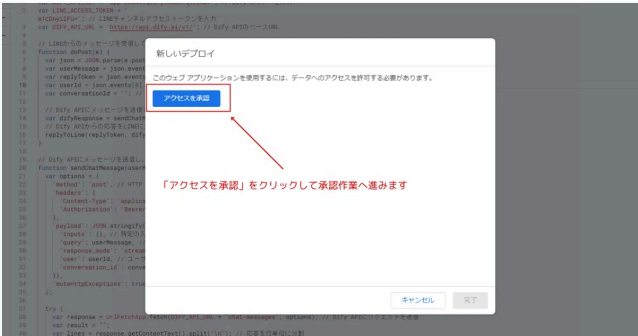
- ・新しい説明文：Dify_Agent_LINE(自分が把握できれば何でもOK)
- ・次のユーザーとして実行：自分
- ・アクセスできるユーザー：全員(必ず全員にしてください)

※アクセスできるユーザーを「全員」にしないと、正常に動作しないので注意してください

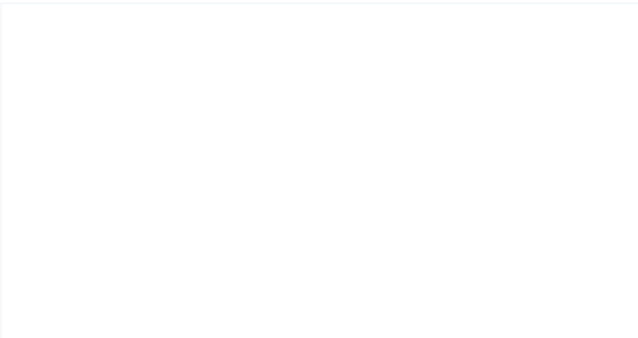
最後に「デプロイ」をクリック



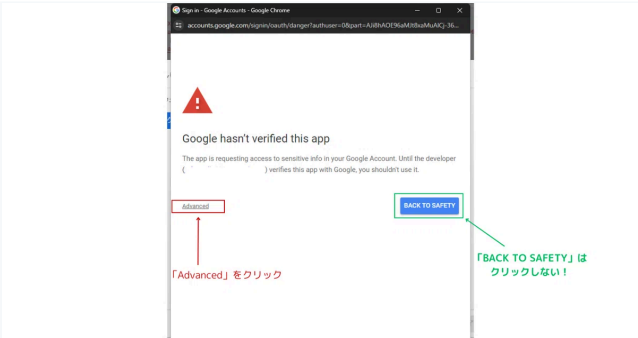
次に「アクセスを承認」をクリックして認証作業を行います。



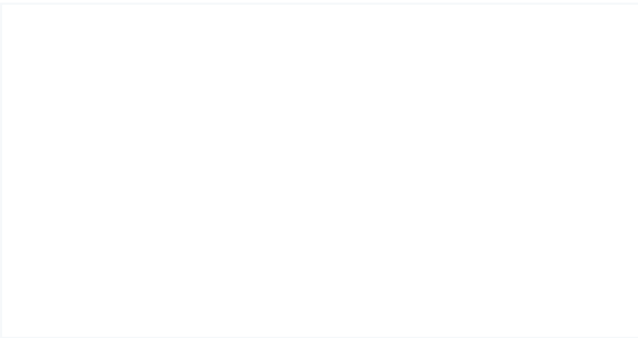
ご自身のアカウントを選択



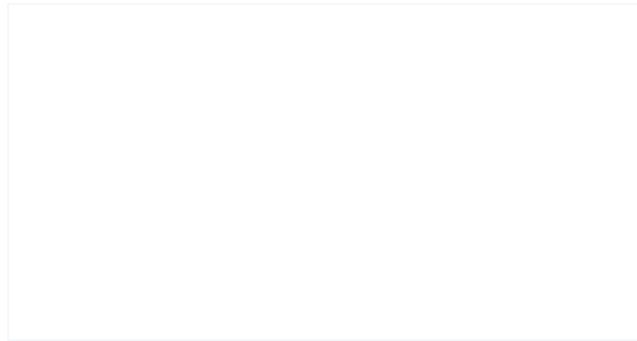
「Advanced」をクリック
※「BACK TO SAFETY」はクリックしない！



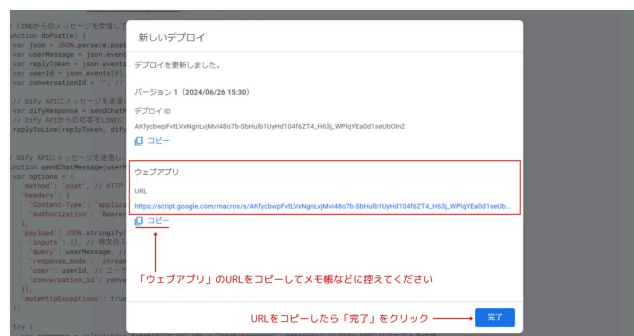
「Go To Dify_Agent_LINE」をクリック
※ここでも「BACK TO SAFETY」はクリックしない！



「Allow」をクリック



「ウェブアプリ」のURLをコピーしてメモ帳などに控えてください
URLをコピーしたら「完了」をクリック



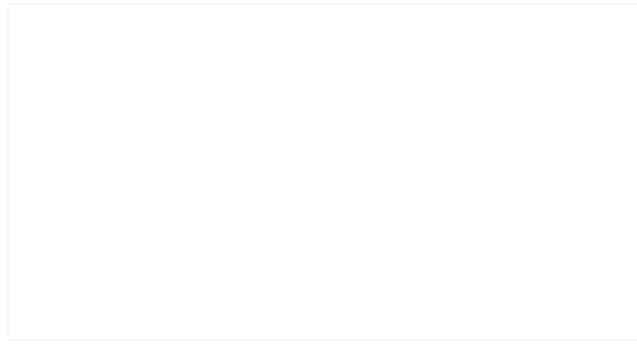
「ウェブアプリ」のURLを控えたら、4.GASコードにAPIをはめ込み・デプロイの設定は終了です。

5. LINEとGASの連携

最後にLINEとGASを連携させます。
再度**LINE Developers**に移動します。

5-1. Webhookの設定

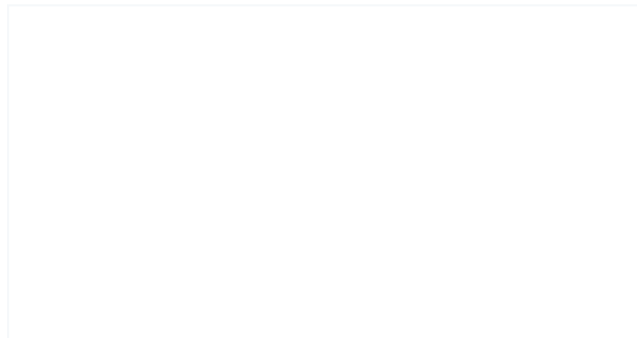
Messaging API設定のページのQRコード下にある「Webhook設定」の「編集」をクリック



「編集」をクリックするとURL入力欄が出ますので、先程デプロイした際に控えた「ウェブアプリ」のURLを入力

※URLの最後が「/exec」のものになります

WebhookにURLを入力したら「更新」をクリック

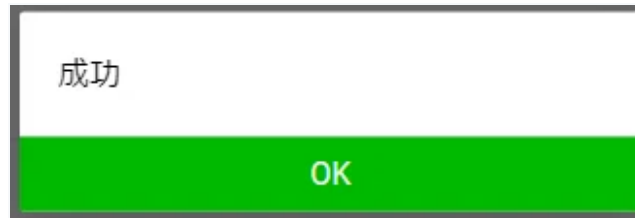


「更新」をクリックしたらWebhook URLの下にある「Webhookの利用」のトグルをクリックして利用をオン(ライム色)にしてください

その後、トグルの上にある「検証」をクリックして「成功」と表示されるとLINEとGASが正常に繋がっています。

※ 設定してすぐは、なぜか成功しているのに「エラー」と表示されることがあります。デプロイした際のウェブアプリURLをきちんと入力したにも関わらず、何度やってもエラーの場合は、一度先に進んでみてください。



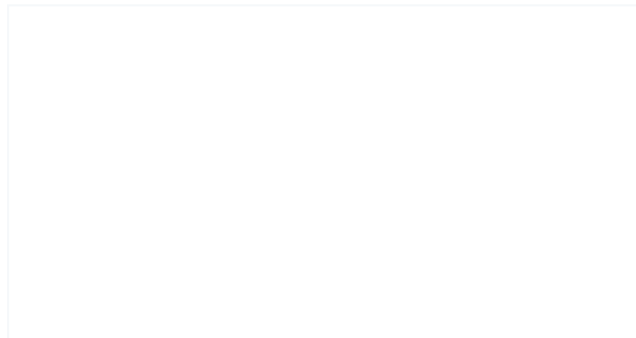


5-2. トークの設定

次にLINEの自動返信やチャットの設定を行います。

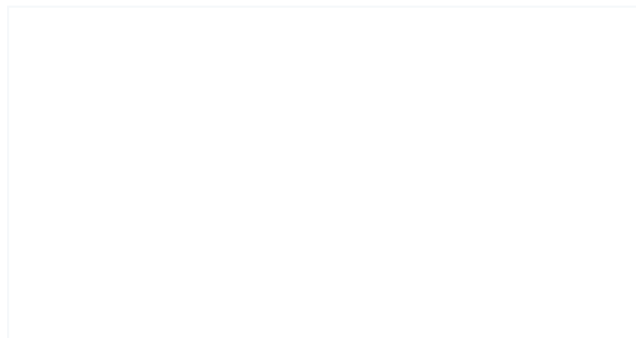
現時点だと、「応答メッセージ」・「あいさつメッセージ」が「有効」になっていると思いますが、この2つを「無効」にします。

「応答メッセージ」の右側にある「編集」をクリック



「編集」をクリックすると自動で**LINE Official Account Manager**に飛びますので、「応答機能」を以下のように設定します

- ・チャット：**オフ**
- ・あいさつメッセージ：**オフ**
- ・Webhook：**必ずオン！**
- ・応答メッセージ：**オフ**
- ・応答時間：**オフ**



以上で5.LINEとGASの連携は終了になります

これでDifyエージェントとLINEの連携は完了になります。

ご自身の携帯電話のLINEから、LINE Developersの「Messaging API」にあるQRコードを読み込んで、友だちに追加することで使用できます。

Difyで作成したエージェントをLINEに連携させることで気軽に使用することができ、時短になります。

また、ビジネスでも大きな武器になるかもしれません。

Difyで様々なエージェントを作成し、LINEと連携させて自分好みにカスタマイズさせてみましょう。

「AIを使ってシステムを構築したい」という方へ

AIを使えば、今回のような構築をもっと短時間で設定することが可能になります。それが、いま話題の『AI駆動開発』です。

私たちは、このAI駆動開発を実践的に学べる

『AI駆動開発ハンズオン講座』

の参加者を募集しています。

この講座は

GASプロダクト全般を開発できるようになるスキルセットを習得し

3週間でプロダクト完成をめざす

実践講座です。

【学べること】

- ✓ AI駆動型GASプロダクトの設計～実装まで
- ✓ Clasp、GCP、Firebase、CI/CDを実務レベルでマスター
- ✓ Gmail自動返信やGoogle Meet議事録生成など実際に動くプロダクトを完成

気になる方は、以下のリンクから詳細をチェックしてみてください。

AI BOOTCAMP - 生成AIを使える人材へと育てる、実践型AIビジネススクール

AI BOOTCAMPは、生成AIの基礎からオリジナルGPTモデルの開発まで、4日間の集中コースで学べる実践型AIビジネス

www.ai-bootcamp.tokyo